

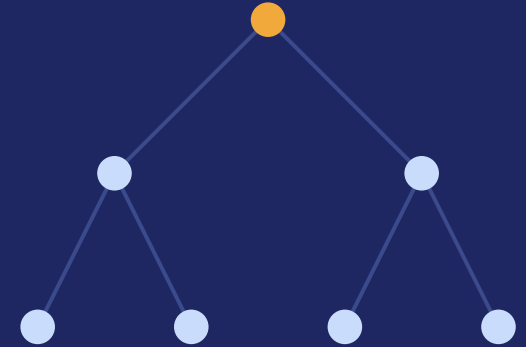
TOPIC 6 · SYNTAX ANALYSIS (PARSING)

ParseLens

An Interactive Web-Based Syntax Analyzer & Parse Tree Visualizer

Compiler Design (CSA14) · Assessment Tool 3

Presented by Athavan k (192524036)



What Syntax Analysis Is — and Why It Matters

1

Second Phase of Compilation

Syntax analysis (parsing) takes the token stream from the lexer and checks whether it follows the grammatical rules of the language, defined by a context-free grammar (CFG).

2

Builds the Parse Tree

A parser (LL or LR) arranges tokens into a hierarchical parse tree that mirrors the grammar's structure, exposing exactly how an expression or statement is composed.

3

Detects Syntax Errors

Any token sequence that cannot be derived from the grammar is reported as a syntax error, with the position pinpointed — catching mistakes before later compiler phases.

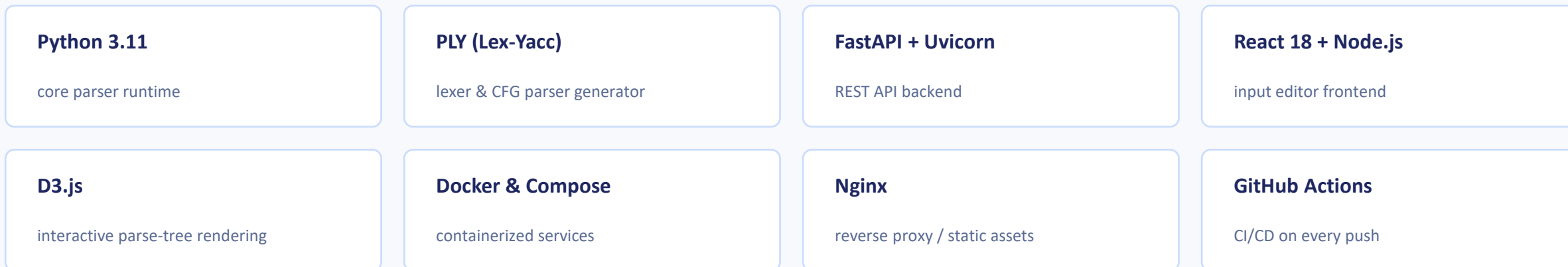
Project goal: build ParseLens, a browser-based tool where users type an expression or statement and watch tokenization, parsing, and parse-tree construction happen live.

Deploying ParseLens as a Web Application

Deployment Pipeline



Required Software & Packages



Limitations in Existing Syntax-Analysis Tools

Command-line only

Classic tools like YACC/Bison run in a terminal and print raw grammar conflicts — no visual parse tree for learners.

Poor error messages

Generic “syntax error” output rarely names the expected token or grammar rule that failed.

No live feedback

Most parser generators require a full compile-run cycle before any syntax error is visible; nothing highlights the bad token as you type.

No tree exploration

Existing tools rarely let users expand/collapse nodes or trace how a token sequence maps to the grammar.

Steep learning curve

Understanding LL vs LR parsing, grammar ambiguity, and derivations is hard without a visual, step-by-step view.

What ParseLens adds

A browser-based analyzer that tokenizes, parses, and renders an interactive parse tree with precise, live error highlighting.

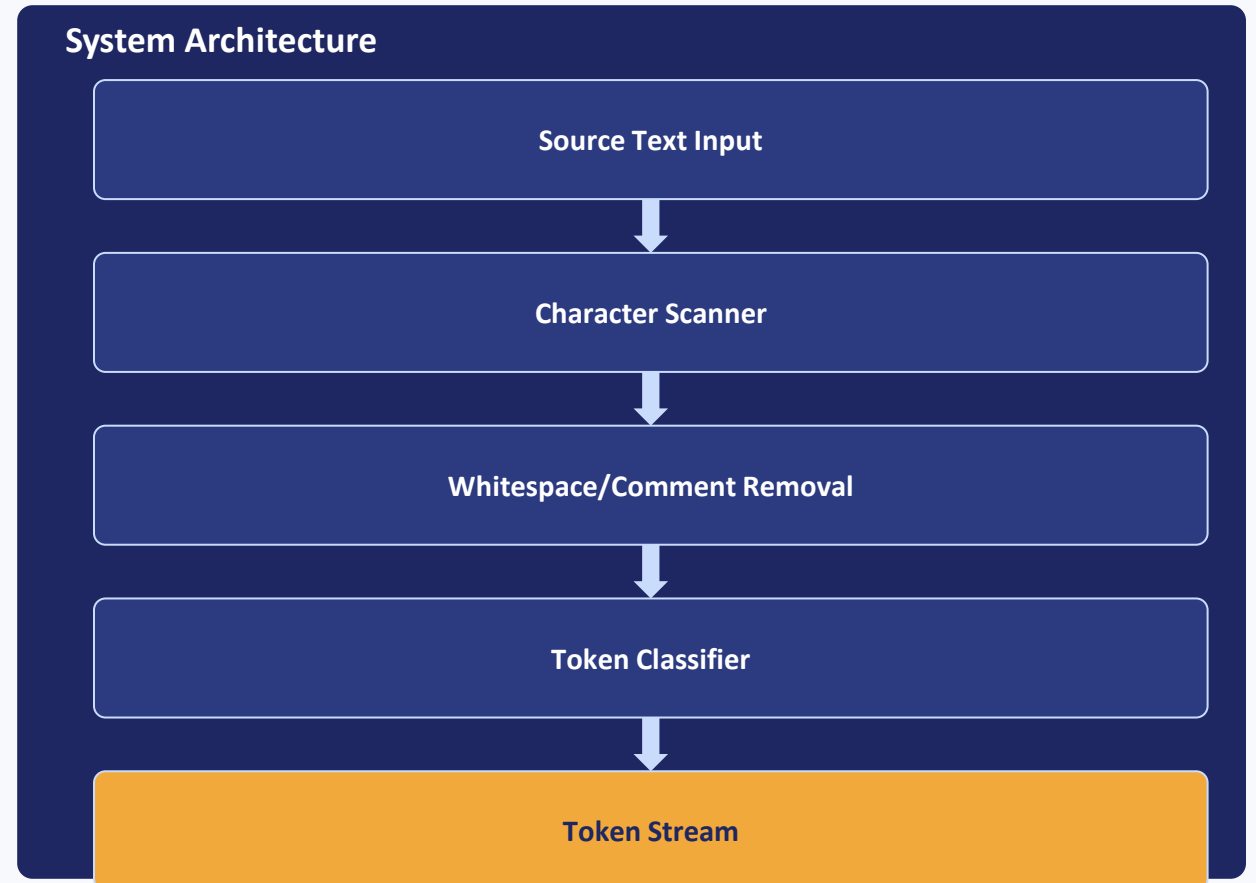
Lexical Analyzer (Tokenizer)

Converts raw source text into a clean stream of tokens for the parser to consume.

Implementation Details

- Regex-based scanner reads characters and groups them into tokens (identifiers, numbers, operators, keywords)
- Strips whitespace and comments before tokens reach the parser
- Assigns each token a type and lexeme, plus line/column position for error reporting
- Reports lexical errors (invalid characters) separately from syntax errors
- Built with Python and PLY's lex module

System Architecture



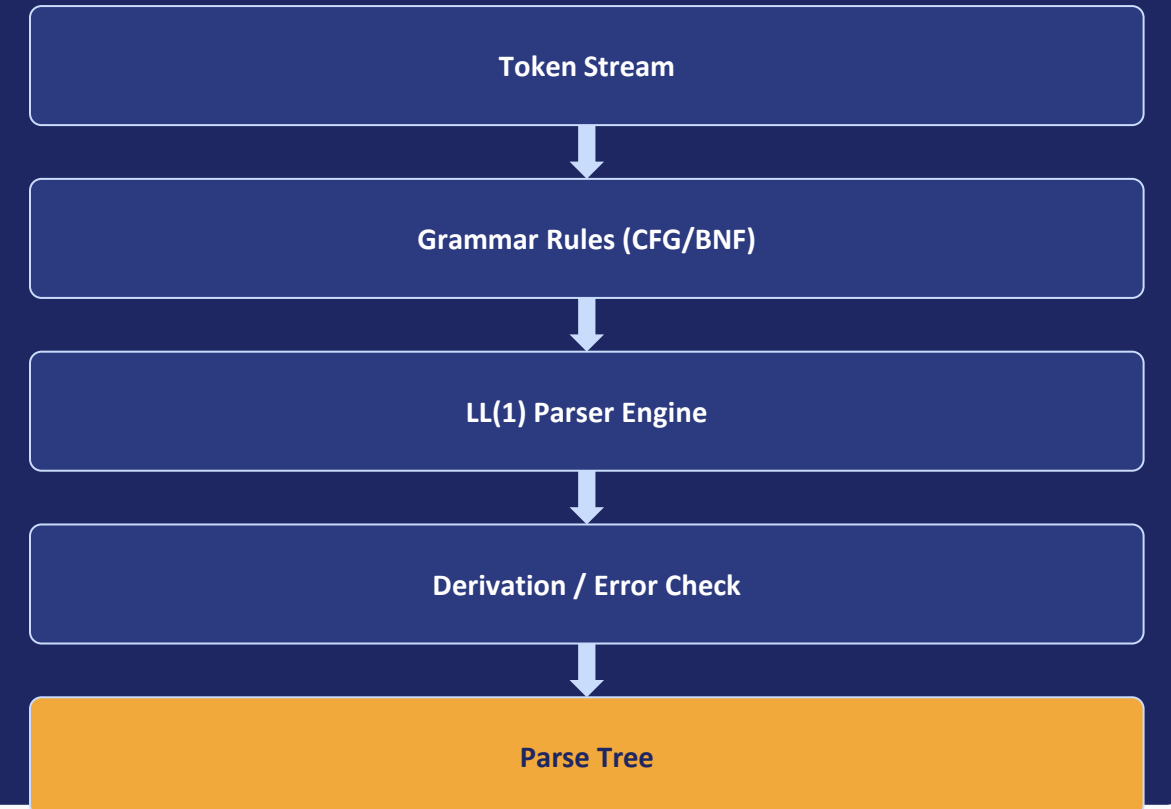
CFG-Based Syntax Analyzer (Parser)

The core module: validates token order against a context-free grammar and builds the parse tree.

Implementation Details

- Grammar defined in Backus-Naur Form (BNF) for the supported expression/statement syntax
- LL(1) recursive-descent parser (with an LR/YACC alternative) consumes the token stream
- Applies grammar production rules to derive a parse tree matching the input
- Detects syntax errors the moment a token doesn't match any valid production, reporting the expected token
- Supports operator precedence and nested expressions via grammar recursion

System Architecture



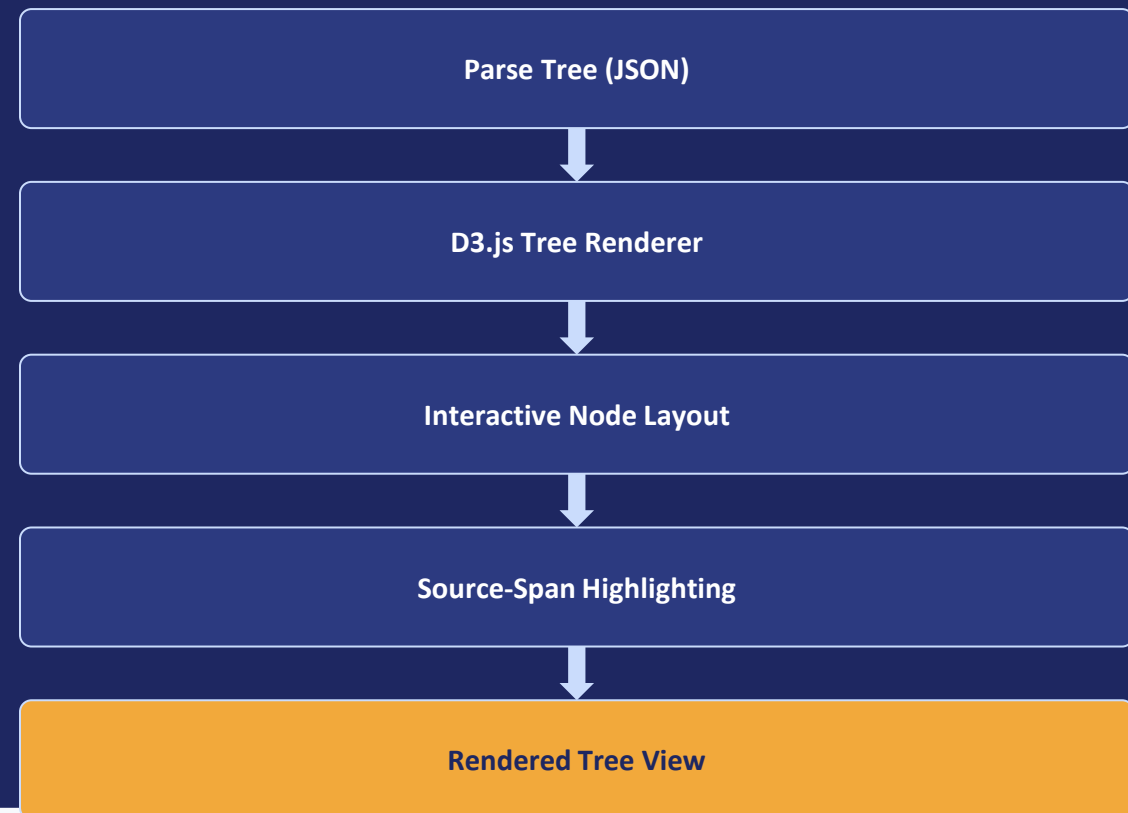
Parse Tree Visualizer

Renders the parser's output as an interactive, explorable tree in the browser.

Implementation Details

- Parse tree serialized to JSON (node type, children, source span) for the frontend
- D3.js renders the tree with expandable/collapsible nodes
- Clicking a node highlights the matching text span in the source editor
- Color-codes node types (terminals vs. non-terminals) for readability
- Supports zoom and pan for larger parse trees

System Architecture



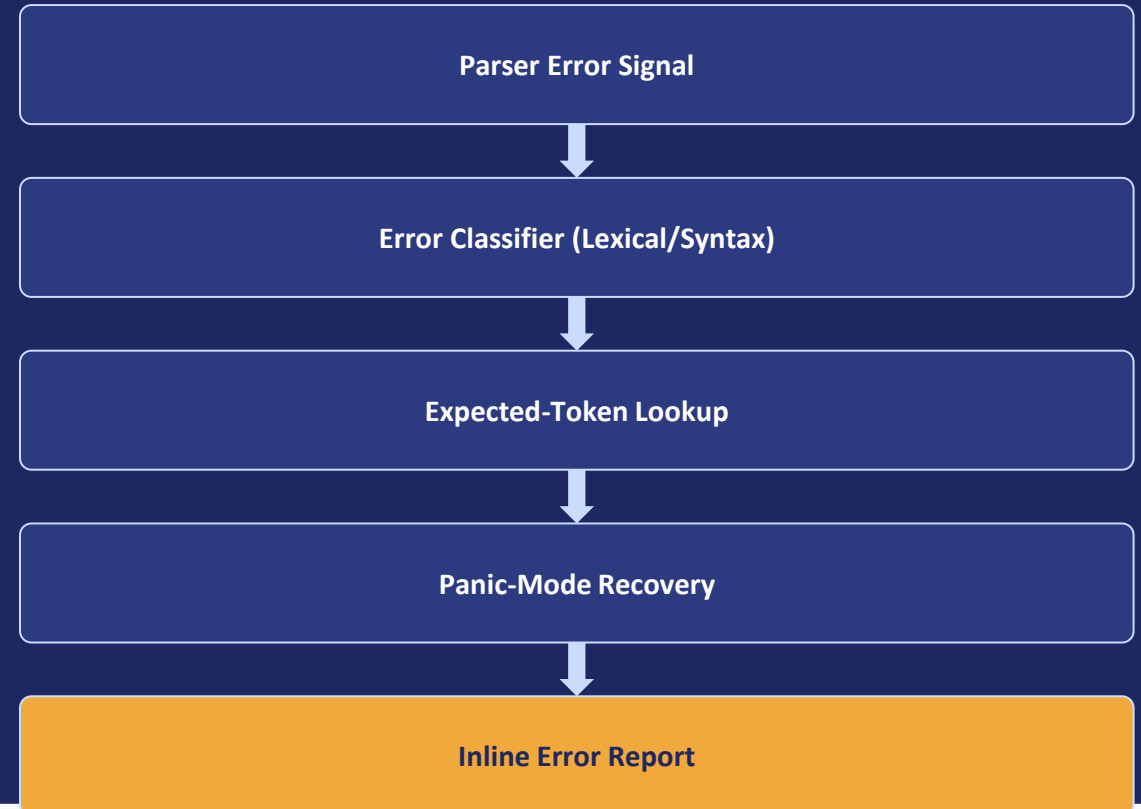
Error Detection & Reporting

Surfaces precise, actionable syntax error messages instead of generic failures.

Implementation Details

- Catches lexical errors (illegal characters) and syntax errors (invalid token order) separately
- Reports the exact line, column, and offending token for every error
- States which token(s) the grammar expected at that point, not just “error”
- Implements panic-mode recovery to keep parsing and surface multiple errors per run
- Displays errors inline in the web editor, underlining the offending text

System Architecture





GPS Map Camera

Mevalurkuppam, Tamil Nadu,
India 🇮🇳

22g8+cwh, Mevalurkuppam, Tamil Nadu 602105,
India

Lat 13.02645° Long 80.017172°

Thursday, 30/07/2026 11:06 AM GMT +05:30

Google